

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

Digital Filter design and Implementation

Presented By:

Huzaifa Shahbaz (21-CE-009)

Outlines of Digital filter design

- Abstract
- Introduction
- Objective
- Expected Outcome
- Methodology
- Component used
- Digital Filter Design
- Finite Impulse Response

Outlines of Digital filter design(continued)

- Infinite Impulse Response
- Functional Requirements
- Frequency Response Specifications
- Filter Type Selection
- Filter Order and Complexity
- Stability and Robustness
- Transition Bandwidth and Ripple

Outlines of Digital filter design(continued)

- Phase Linearity
- Implementation Constraints
- Filter Optimization
- Filtering Characteristics
- Validation and testing
- Non-functional Requirements
- Performance
- Scalability

Outlines of Digital filter design(continued)

- Reliability
- Robustness
- Compatibility
- Maintainability
- Resource Efficiency
- Usability
- Portability
- Compliance

Outlines of Digital filter design(continued)

- Code
- Kaiser window
- Original Signal, Noisy Signal and Filtered Signal
- Frequency Filtered Response
- Conclusion

Abstract

- In abstract, we discuss the digital filter design and implementation, distinguishing between finite impulse response(FIR) and infinite impulse response(IIR) filters and highlighting key design methods like windowing and optimization algorithms.
- It highlights the use of software tools and their applications in audio, communications, and image processing.

Introduction

- Developing algorithms to process digital signals and eliminate undesirable elements or enhance desired aspects while maintaining signal characteristics is known as digital filter design.
- The impulse response of IIR filters is unlimited, whereas that of FIR filters is finite.
- Windowing, frequency sampling, optimization techniques, and software tools are all used in implementation.
- Image processing, telecommunication, and audio processing are examples of applications.

Objective

- This technology improves signal quality, achieves desired frequency response, optimizes computational efficiency, ensures stability, facilitates parameter tuning, minimizes complexity, and integrates seamlessly into hardware and software platforms for practical deployment.

Expected Outcome

- The goal of digital filter design and implementation is to use mathematical methods to obtain desired frequency response characteristics, such as low-pass, high-pass, band-pass, or band-stop.
- The expected result is a filter with the desired frequency domain features, such as attenuation or amplification, within specified passbands and stopbands.
- Implementation often entails programming the filter algorithm in a digital signal processing (DSP) platform or hardware to ensure efficient calculation and real-time performance.
- The end product should show efficient suppression or isolation of specific frequencies while preserving others, thereby improving signal quality or extracting desired information.

Expected Outcome(Continued)

- Testing and validation compare the filter's performance to design specifications, verifying that it is appropriate for its intended uses, such as audio processing, image enhancement, or communication systems.
- Continuous refining and optimization can be performed to improve filter efficiency and minimize computational complexity, or adapt to changing requirements.

Methodology

- To design a digital filter, define filter specifications, choose a filter type, design an analog prototype, apply frequency transformation, choose sampling frequency and discretization method, implement the digital filter, and verify performance through simulation and testing.

Component used

- The components used of the digital filter design and implementation are following.
- Spyder
- Visual Studio Code

Digital Filter Design

- A digital filter uses a digital processor to perform numerical calculations on sampled values of the signal.
- The processor may be a general-purpose computer such as a PC, or a specialized DSP (Digital Signal Processor) chip.
- There are the following types of digital filter design are
 - Finite Impulse Response
 - Infinite Impulse Response

Finite Impulse Response

- Finite Impulse Response (FIR) filters are a type of digital filter characterized by a finite duration of their impulse response, settling to zero in a finite number of steps.
- They exhibit linear phase response, making them suitable for applications requiring phase preservation.
- Implementation involves convolution of the input signal with the filter coefficients.
- The computational efficiency can be optimized using Fast Fourier Transform (FFT) techniques.
- Validation ensures the filter meets performance criteria, providing the expected frequency response and minimal distortion.

Infinite Impulse Response

- Infinite Impulse Response (IIR) filters are digital filters whose impulse response is theoretically infinite.
- They are created using analog filter design approaches such as Butterworth, Chebyshev, or elliptic filters, and then converted to the digital domain using bilinear transformation or matching z-transform.
- IIR filters use fewer coefficients than FIR filters to obtain comparable frequency response characteristics, making them more computationally efficient.
- However, feedback can cause instability and nonlinear phase response.

Infinite Impulse Response(Continued)

- Implementation entails recursive algorithms with outputs based on both current and previous input and output values.
- Proper design and validation are required to assure stability and compliance with specified criteria.
- Applications include audio processing, control systems, and communication.

Functional Requirements of digital filter design

- The functional requirements of digital filter design are.
- **Frequency Response Specifications**
- **Filter Type Selection**
- **Filter Order and Complexity**
- **Stability and Robustness**
- **Transition Bandwidth and Ripple**
- **Phase Linearity**
- **Implementation Constraints**
- **Filter Optimization**
- **Filtering Characteristics**
- **Validation and testing**

Non-functional Requirements

- The non-functional requirements of digital filter design are
- **Performance**
- **Scalability**
- **Reliability**
- **Robustness**
- **Compatibility**
- **Maintainability**
- **Resource Efficiency**
- **Usability**
- **Portability**
- **Compliance**

Code of digital filter design

- `import numpy as np`
- `import matplotlib.pyplot as plt`
- `from scipy.signal import hamming, lfilter, freqz`
- `np.random.seed(0) # For reproducibility`
- `fs = 700 # Sampling Frequency`
- `t = np.linspace(0, 1.0, fs) # Time vector`
- `freq1 = 10 # Frequency of the first sine wave`
- `freq2 = 80 # Frequency of the second sine wave`
- `clean_signal = np.sin(2 * np.pi * freq1 * t) + np.sin(2 * np.pi * freq2 * t)`
- `# Add noise`
- `noise = np.random.normal(0, 0.5, clean_signal.shape)`
- `noisy_signal = clean_signal + noise`

Code of digital filter design(continued)

- # Step 2: Design the digital filter (low-pass FIR filter using window method)
- `cutoff = 10` # Desired cutoff frequency of the filter, Hz
- `nyquist = 0.5 * fs` # Nyquist frequency
- `normal_cutoff = cutoff / nyquist` # Normalized cutoff frequency
- `numtaps = 101` # Size of the FIR filter
- # Use a Hamming window to design the filter
- `window = hamming(numtaps)`
- `taps = np.sinc(2 * normal_cutoff * (np.arange(numtaps) - (numtaps - 1) / 2))`
`*window`
- `taps /= np.sum(taps)` # Normalize the filter coefficients

Code of digital filter design(Continued)

- # Step 3: Apply the filter to the noisy signal
- `filtered_signal = lfilter(taps, 1.0, noisy_signal)`
- # Step 4: Plot the results
- `plt.figure(figsize=(15, 10))`
- # Original signal
- `plt.subplot(3, 1, 1)`
- `plt.plot(t, clean_signal)`
- `plt.title('Original Signal')`
- `plt.xlabel('Time [seconds]')`
- `plt.ylabel('Amplitude')`

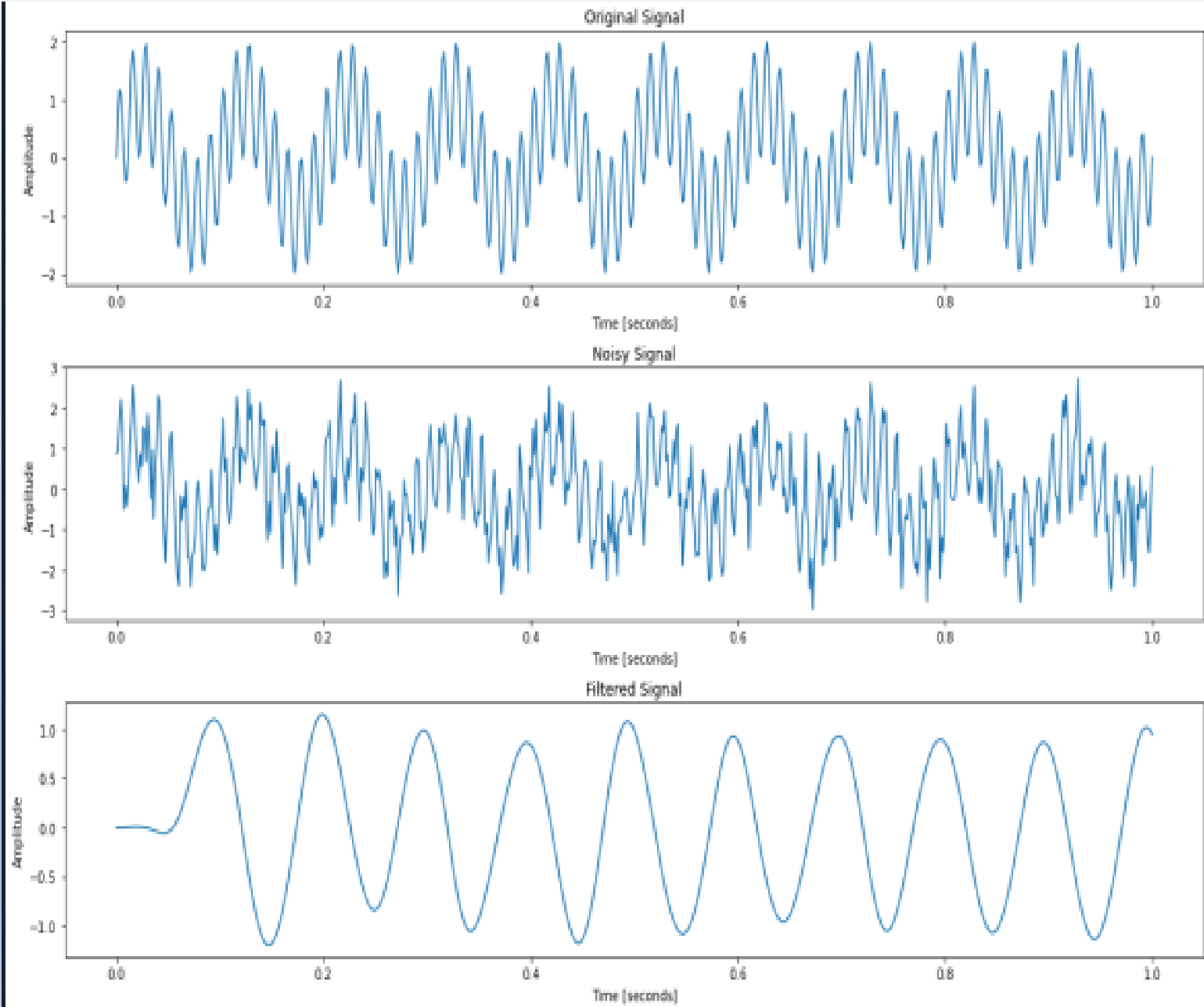
Code of digital filter design(Continued)

- # Plot frequency response of the filter
- `w, h = freqz(taps, worN=8000)`
- `plt.figure()`
- `plt.plot(0.5 * fs * w / np.pi, np.abs(h), 'b')`
- `plt.title('Filter Frequency Response')`
- `plt.xlabel('Frequency [Hz]')`
- `plt.ylabel('Gain')`
- `plt.grid()`
- `plt.show()`

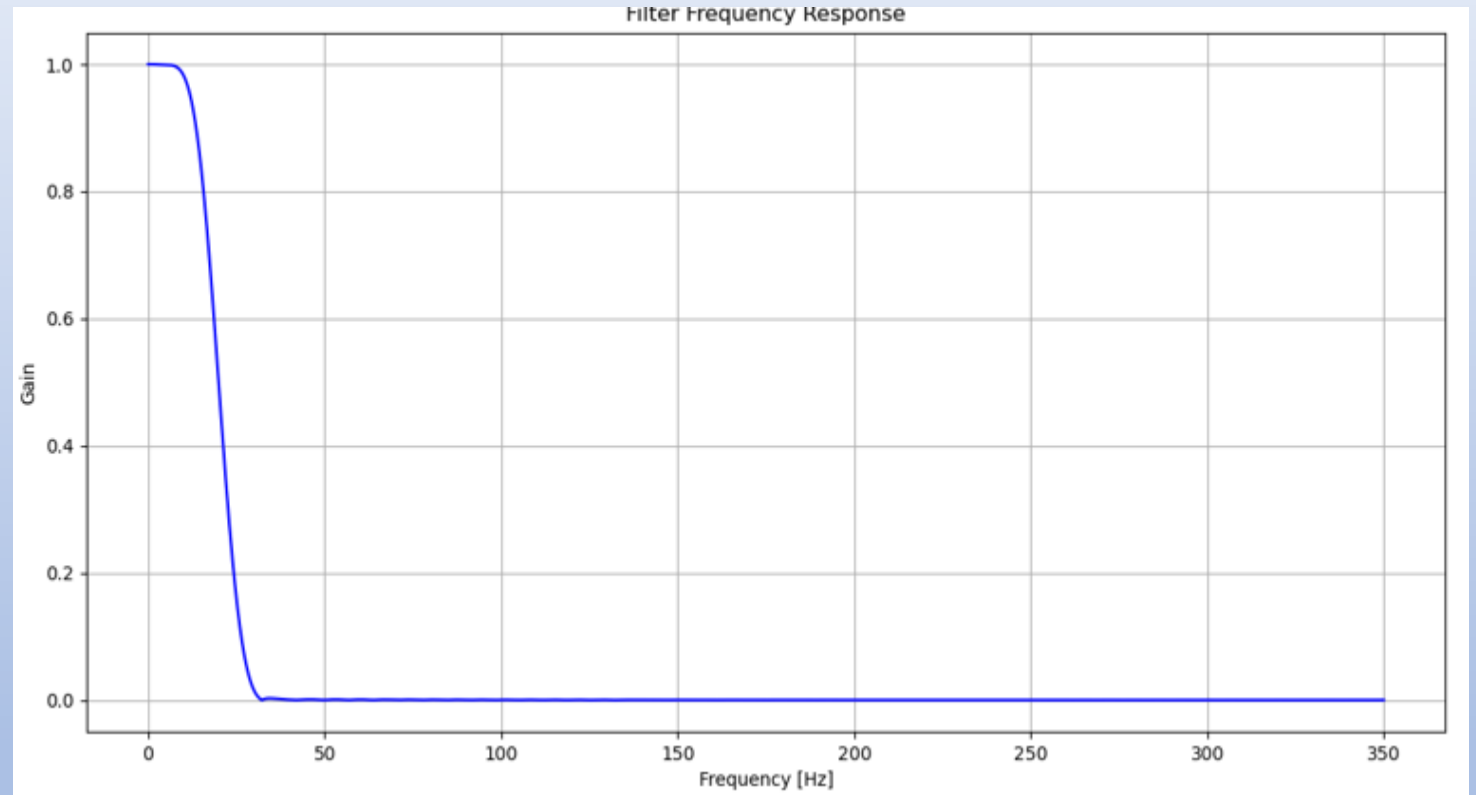
Kaiser Window

- It is a one-parameter family of window functions used in finite impulse response filter design and spectral analysis.
- The Kaiser window approximates the Diode Pumped Solid State (DPSS) window which maximizes the energy concentration in the main lobe but which is difficult to compute.

Results and graphs Original Signal, Noisy Signal and Filtered Signal



Results and graphs Filter Frequency Response



Conclusion

- In conclusion, this work gave a thorough approach to digital filter design, including both theoretical and practical aspects.
- The approach used to develop both Finite Impulse Response (FIR) and Infinite Impulse Response (IIR) filters was described, with a focus on their functional and non-functional needs.
- The Kaiser Window approach was recognized as an efficient way to improve filter performance.
- Several tests were conducted to confirm the implementation, demonstrating the filter's capacity to effectively reduce noise while keeping signal integrity.
- The findings and graphs gave a clear visual representation of the filter's performance, verifying the design's success.